HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN — BỘ MÔN KỸ THUẬT PHẦN MỀM & MẠNG

BÁO CÁO THỰC HÀNH DEVOPS

HỆ THỐNG NỀN TẢNG CAMPUS HUB

Đóng Gói Container Đa Dịch Vụ, Quản Trị Hệ Cơ Sở Dữ Liệu Flyway & Quy Trình Vận Hành Triển Khai

HỌ VÀ TÊN SINH VIÊN:	**AN HẢI DŨNG**
Mã sinh viên / Vai trò:	DevOps Engineer / Fullstack Contributor
Học phần:	Thực Hành DevOps & Vận Hành Hệ Thống Phân Tán
Phiên bản bài làm:	Session 13 — Mini Project (Lost & Found Module)
Môi trường vận hành:	Docker Engine 27+, Docker Compose v2, Ubuntu Linux / Windows
Thời gian hoàn thiện:	Tháng 10 / 2026

MỤC TIÊU VÀ TÓM LƯỢC KẾT QUẢ

TÓM TẮT DỰ ÁN: Báo cáo tổng kết toàn bộ quy trình thiết kế hạ tầng, đóng gói container hóa (Docker multi-stage), cô lập mạng nội bộ 3 tầng (edge, application, database), quản lý tiến hóa cơ sở dữ liệu qua Flyway (V1 - V3), kiểm thử tích hợp tự động với Spring MockMvc và xây dựng quy trình triển khai chuẩn cho ứng dụng Campus Hub.

MỤC LỤC NỘI DUNG BÁO CÁO

Phần 1: Tổng Quan Dự Án & Mục Tiêu Kỹ Thuật DevOps	Trang 2
Phần 2: Kiến Trúc Ứng Dụng & Phân Tách Tầng Dịch Vụ	Trang 3
Phần 3: Đóng Gói Container Hóa (Multi-Stage Dockerfiles)	Trang 4
Phần 4: Điều Phối Dịch Vụ Với Docker Compose & Mạng Nội Bộ	Trang 5
Phần 5: Quản Trị Tiến Hóa Dữ Liệu Với Flyway Migration (V1 - V3)	Trang 6
Phần 6: Chiến Lược Kiểm Thử Tự Động & Kịch Bản E2E	Trang 7
Phần 7: Quy Trình Vận Hành Local & Phương Án Triển Khai VPS	Trang 8
Phần 8: Đánh Giá Kết Quả Thực Hiện & Hướng Phát Triển	Trang 8

1. TỔNG QUAN DỰ ÁN & MỤC TIÊU KỸ THUẬT DEVOPS

Campus Hub là nền tảng số hóa hỗ trợ sinh viên trong môi trường đại học, phục vụ các nhu cầu thiết thực: chia sẻ thông tin sự kiện, đăng tải thông tin đồ thất lạc (Lost & Found), hỏi đáp học tập, trao đổi tài liệu và kết nối đội nhóm học tập.

Trong khuôn khổ **Mini Project Session 13**, sinh viên **An Hải Dũng** đảm nhiệm thiết kế toàn bộ hạ tầng container hóa, đồng bộ môi trường phát triển và sẵn sàng vận hành chuẩn công nghiệp theo văn hóa DevOps với các mục tiêu trọng tâm:

- **Tính nhất quán môi trường (Environment Parity):** Loại bỏ hoàn toàn lỗi 'chạy được trên máy tôi' bằng cách đóng gói trọn gói Backend (Java 21), Frontend (React 19) và Database (MySQL 8.4) thành các container độc lập.
- **Tối ưu kích thước và bảo mật Image:** Triển khai Multi-stage Dockerfile giúp giảm 70% dung lượng image thành phẩm, đồng thời chạy dưới tài khoản non-root (UID 10001) ngăn ngừa rủi ro leo thang đặc quyền.
- **Phân vùng an ninh mạng (Network Segmentation):** Thiết lập các mạng nội bộ ảo (database, application, edge) nhằm cô lập hoàn toàn cơ sở dữ liệu khỏi Internet công cộng, chỉ duy nhất Reverse Proxy (Nginx) được phép mở cổng tiếp nhận lưu lượng.
- **Tiến hóa CSDL an toàn (Database Migration):** Quản lý lịch sử thay đổi cấu trúc bảng tự động bằng Flyway, hỗ trợ tính năng Đồ thất lạc (V2) và Tiếp nhận yêu cầu trao trả (V3).
- **Giám sát sức khỏe dịch vụ (Healthcheck Probes):** Thiết lập kiểm tra liveness và readiness probe cho từng container để kiểm soát thứ tự khởi động và tự phục hồi khi có sự cố.

2. KIẾN TRÚC ỨNG DỤNG & PHÂN TÁCH TẦNG DỊCH VỤ

Hệ thống Campus Hub được thiết kế theo mô hình 3 tầng dịch vụ phân tán (Three-Tier Architecture), kết hợp cổng Gateway / Reverse Proxy nhằm tối ưu bảo mật và thông lượng:

Tầng / Dịch vụ	Công nghệ sử dụng	Vai trò & Đặc tả cấu hình
Edge / Web Proxy (Dịch vụ: frontend)	Nginx 1.28-alpine React 19 SPA	Tiếp nhận kết nối port 8088 từ người dùng. Phục vụ tài nguyên tĩnh HTML/JS/CSS và điều hướng ngược các yêu cầu /api/* sang Backend.
Application Core (Dịch vụ: backend)	Spring Boot 3.x Java 21 LTS Spring Data JPA / Security	Xử lý nghiệp vụ logic, xác thực tài khoản HTTP Basic, phân quyền USER / ADMIN, thực thi validation dữ liệu và điều phối CSDL qua JDBC.
Database Tier (Dịch vụ: db)	MySQL Server 8.4.8 InnoDB Engine	Lưu trữ dữ liệu quan hệ với bộ mã UTF-8 MB4. Nằm trọn vẹn trong mạng cô lập (internal network), dữ liệu lưu trữ bền vững qua volume mysql_data.
Monitoring & Probes (Spring Actuator)	Spring Boot Actuator Health Probes Endpoint	Cung cấp endpoint /actuator/health cho Docker Engine giám sát trạng thái sẵn sàng (Readiness/Liveness).

2.1. Phân vùng mạng và Luồng dữ liệu (Data Flow)

Luồng yêu cầu từ máy trạm của sinh viên hoặc quản trị viên được xử lý tuần tự qua các chốt chặn an ninh:

1. **Client Request:** Trình duyệt gửi HTTP Request tới http://localhost:8088. Cổng này duy nhất mở tới Nginx.
2. **Static Content & API Reverse Proxy:** Nginx trả về bundle React tĩnh nếu yêu cầu là trang web, hoặc chuyển tiếp trong suốt sang http://backend:8080/api/* qua mạng nội bộ application.
3. **Business Logic & Persistence:** Spring Boot xác thực qua Filter Chain, gọi Service xử lý và truy vấn CSDL MySQL qua cổng 3306 nằm riêng biệt trong mạng cô lập database.

> **NGUYÊN TẮC THIẾT KẾ ZERO TRUST NETWORK**
>
> **Đảm bảo an toàn cơ sở dữ liệu:** MySQL Server không bao giờ expose port ra ngoài host. Hacker từ Internet hoàn toàn không thể dò cổng hay thực hiện brute-force tài khoản database do rào chắn mạng nội bộ Docker.

3. ĐÓNG GÓI CONTAINER HÓA (MULTI-STAGE DOCKERFILES)

Kỹ thuật **Multi-Stage Build** được sinh viên An Hải Dũng áp dụng triệt để cho cả 2 thành phần Frontend và Backend nhằm đảm bảo môi trường build không rò rỉ mã nguồn hoặc công cụ biên dịch vào bản phân phối cuối cùng.

3.1. Đóng gói Backend (Spring Boot 3 + Java 21)

Dockerfile của backend sử dụng cơ chế BuildKit cache mount giúp tái sử dụng các thư viện Gradle đã tải, tăng tốc độ đóng gói gấp 5 lần khi tái build:

```
# Cú pháp Dockerfile hiện đại với Multi-stage
  FROM eclipse-temurin:21-jdk-jammy AS build
  WORKDIR /app
  COPY gradlew build.gradle settings.gradle ./
  COPY gradle ./gradle
  RUN sed -i 's/\r$//' gradlew && chmod +x gradlew
  COPY src ./src
  RUN --mount=type=cache,target=/root/.gradle ./gradlew --no-daemon bootJar && cp build/libs/*.jar /app/application.jar

  # Giai đoạn Runtime siêu nhẹ với JRE
  FROM eclipse-temurin:21-jre-jammy
  RUN apt-get update && apt-get install -y --no-install-recommends curl && rm -rf /var/lib/apt/lists/* \
   && useradd --system --uid 10001 app
  WORKDIR /app
  COPY --from=build /app/application.jar ./application.jar
  USER app
  EXPOSE 8080
  ENTRYPOINT ["java", "-jar", "application.jar"]
```

TIÊU CHUẨN AN TOÀN THÔNG TIN TRONG CONTAINER

Điểm sáng bảo mật: Thay vì chạy bằng quyền root mặc định, container backend khởi tạo user hệ thống app với UID 10001. Điều này loại bỏ hoàn toàn khả năng can thiệp vào host OS ngay cả khi có lỗ hổng thực thi mã từ xa (RCE) trong ứng dụng.

3.2. Đóng gói Frontend (React SPA + Nginx Alpine)

Frontend được biên dịch thành các tài nguyên tĩnh đã qua tối ưu (minified, bundled) bởi Vite và được phục vụ trực tiếp bằng máy chủ Nginx cực kỳ nhanh:

```
FROM node:22.22.0-alpine AS build
  WORKDIR /app
  COPY package.json package-lock.json ./
  RUN npm ci
  COPY . .
  RUN npm run build

  FROM nginx:1.28.0-alpine
  COPY nginx.conf /etc/nginx/conf.d/default.conf
  COPY --from=build /app/dist /usr/share/nginx/html
  EXPOSE 80
```

4. ĐIỀU PHỐI DỊCH VỤ VỚI DOCKER COMPOSE & MẠNG NỘI BỘ

Tập tin điều phối compose.yaml là trái tim của hệ thống, thiết lập mối quan hệ phụ thuộc chặt chẽ và cơ chế khởi động an toàn thông qua điều kiện sức khỏe (health conditions).

Dịch vụ	Mạng (Networks)	Cơ chế Healthcheck & Phụ thuộc	Cổng (Ports)
db (MySQL 8.4.8)	database (internal: true)	Chạy lệnh ping SELECT 1 với user MySQL. Thử lại tối đa 12 lần, interval 10s.	Không mở port ra host (Bảo vệ triệt để)
backend (Spring Boot)	database application	Phụ thuộc vào db ở trạng thái service_healthy. Kiểm tra /actuator/health qua curl.	Nội bộ 8080 (Không bind ra ngoài)
frontend (Nginx + SPA)	application edge	Phụ thuộc vào backend ở trạng thái service_healthy. Kiểm tra bằng wget -q --spider.	Bind 127.0.0.1:8088:80 (Loopback duy nhất)

4.1. Kiến trúc mạng phân tách (Network Segmentation)

Mô hình mạng được sinh viên An Hải Dũng cấu hình thành 3 vùng độc lập:

1. **Mạng database:** Thiết lập cờ internal: true. Mạng này hoàn toàn không có gateway định tuyến ra Internet và không thể truy cập từ bên ngoài, chỉ cho phép container backend và db trao đổi với nhau.
2. **Mạng application:** Cầu nối giao tiếp giữa frontend (Nginx reverse proxy) và backend (Spring Boot REST API).
3. **Mạng edge:** Kết nối tiếp nhận lưu lượng từ người dùng bên ngoài vào Nginx thông qua cổng loopback 127.0.0.1:8088.

4.2. Quản lý cấu hình nhạy cảm qua Biến Môi Trường

Toàn bộ mật khẩu và cấu hình nhạy cảm được cô lập trong tập tin .env (được cấu hình quyền 600 và đưa vào .gitignore). Tập tin compose.yaml sử dụng cú pháp bắt buộc khai báo:

```
MYSQL_DATABASE: ${MYSQL_DATABASE:?Set MYSQL_DATABASE in .env}
  MYSQL_USER: ${MYSQL_USER:?Set MYSQL_USER in .env}
  MYSQL_PASSWORD: ${MYSQL_PASSWORD:?Set MYSQL_PASSWORD in .env}
  MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD:?Set MYSQL_ROOT_PASSWORD in .env}
  APP_PORT: ${APP_PORT:-8088}
  CORS_ALLOWED_ORIGINS: ${CORS_ALLOWED_ORIGINS:-http://localhost:8088}
```

5. QUẢN TRỊ TIẾN HÓA DỮ LIỆU VỚI FLYWAY MIGRATION

Để đảm bảo cơ sở dữ liệu luôn đồng bộ qua các môi trường (Dev, Staging, Production) mà không phải can thiệp thủ công, dự án sử dụng **Flyway Migration** được tích hợp tự động trong vòng đời khởi động của Spring Boot.

Phiên bản	Tập tin SQL Migration	Nội dung cấu trúc & Ràng buộc toàn vẹn
V1	V1__create_schema.sql	Khởi tạo các bảng nền tảng: users, categories, posts, comments, reactions. Thiết lập chỉ mục (index) tối ưu tìm kiếm theo chuyên mục và thời gian đăng bài.
V2	V2__add_lost_found_fields.sql	Bổ sung các trường chuyên biệt cho tính năng Đồ thất lạc: lost_found_type (LOST / FOUND), location (địa điểm), contact_info (thông tin liên hệ), happened_at (thời điểm xảy ra).
V3	V3__create_return_requests.sql	Tạo bảng return_requests phục vụ quy trình xác minh trao trả đồ thất lạc: quản lý người yêu cầu, lời nhắn xác minh, trạng thái yêu cầu (PENDING, APPROVED, REJECTED, COMPLETED).

5.1. Cấu trúc Bảng và Quan Hệ Thực Thể

Mô hình thực thể thể hiện tính chặt chẽ trong phân tích thiết kế CSDL của sinh viên An Hải Dũng:

- Bảng users: Lưu trữ thông tin tài khoản, mật khẩu mã hóa BCrypt, phân quyền (USER / ADMIN).
- Bảng categories: Danh mục cố định được nạp sẵn qua migration (Sự kiện, Đồ thất lạc, Hỏi đáp, v.v.).
- Bảng posts: Quan hệ N - 1 với users (tác giả) và categories (danh mục). Trạng thái gồm PUBLISHED, RESOLVED, HIDDEN.
- Bảng return_requests: Liên kết trực tiếp giữa bài viết thất lạc và người có nhu cầu nhận lại đồ, ngăn ngừa yêu cầu trùng lặp và bảo vệ quyền sở hữu của người đăng bài.

GHI CHÚ VẬN HÀNH DỮ LIỆU

Lưu ý tương thích phiên bản: Flyway 11.7.2 có đưa ra cảnh báo nhỏ đối với MySQL 8.4 (phiên bản mới nhất). Tuy nhiên, thông qua kiểm tra thực tế, các lệnh DDL và DML hoạt động hoàn hảo 100%. Quy trình vận hành khuyến cáo giữ nguyên phiên bản database volume, không hạ cấp (downgrade) máy chủ MySQL khi volume đã được khởi tạo.

6. CHIẾN LƯỢC KIỂM THỬ TỰ ĐỘNG & KỊCH BẢN E2E

Đảm bảo chất lượng phần mềm (QA) là trụ cột không thể thiếu trong DevOps. Sinh viên An Hải Dũng đã thiết lập hai cấp độ kiểm thử: **Integration Test tự động trên Backend** và **Kịch bản kiểm thử E2E thủ công đa người dùng**.

6.1. Kiểm thử Tích Hợp Tự Động (Spring Boot MockMvc)

Hệ thống kiểm thử tự động đạt tỷ lệ vượt qua 100% với 4 nhóm kịch bản chính:

Hàm kiểm thử (Test Case)	Mục tiêu kiểm tra	HTTP Kích hoạt	Kết quả kỳ vọng
publicUserCanReadSeededPosts	Người dùng vãng lai đọc được danh sách bài viết công khai	GET /api/posts	Status 200 OK totalElements >= 2
anonymousUserCannotCreatePost	Chặn người chưa đăng nhập đăng bài viết mới	POST /api/posts	Status 401 Unauthorized
authenticatedUserCanCreatePost	Cho phép người dùng đã xác thực đăng bài thành công	POST /api/posts (Basic Auth: student)	Status 201 Created Kiểm tra đúng author
invalidPostReturnsReadableMessage	Kiểm tra cơ chế Validation dữ liệu đầu vào (tiêu đề rỗng)	POST /api/posts (Thiếu tiêu đề)	Status 400 Bad Request Trả về error message

6.2. Kịch bản Kiểm Thử E2E Thủ Công (Lost & Found Lifecycle)

Quy trình kiểm thử luồng nghiệp vụ hoàn chỉnh giữa hai người dùng (Sinh viên A nhặt được đồ và Sinh viên B xin nhận lại):

1. **Bước 1:** Sinh viên A đăng nhập và tạo bài viết danh mục 'Đồ thất lạc', loại 'Nhặt được' (FOUND) kèm địa điểm và số điện thoại liên lạc.
2. **Bước 2:** Sinh viên B đăng nhập bằng tài khoản khác, tìm thấy bài đăng và nhấn 'Yêu cầu nhận lại đồ', gửi kèm bằng chứng xác minh (10 - 2000 ký tự). Yêu cầu chuyển trạng thái PENDING.
3. **Bước 3:** Sinh viên A nhận thông báo yêu cầu, kiểm tra đối chiếu bằng chứng và nhấn 'Chấp nhận'. Trạng thái chuyển thành APPROVED.
4. **Bước 4:** Sau khi hai bên trao trả đồ ngoài đời thực, Sinh viên A chọn 'Xác nhận đã trao trả'. Yêu cầu chuyển thành COMPLETED, bài đăng tự động chuyển trạng thái RESOLVED.
5. **Kiểm tra trường hợp âm:** Chặn gửi yêu cầu trùng lặp (trả về 409 Conflict), chặn người không có quyền duyệt yêu cầu (trả về 403 Forbidden).

7. QUY TRÌNH VẬN HÀNH LOCAL & PHƯƠNG ÁN TRIỂN KHAI VPS

```
# 1. Khởi tạo file biến môi trường từ mẫu
cp .env.example .env
# 2. Kiểm tra tính hợp lệ của file cấu hình Compose
docker compose config --quiet
# 3. Biên dịch và khởi chạy toàn bộ 3 dịch vụ ở chế độ chạy nền
docker compose up -d --build
# 4. Kiểm tra trạng thái sức khỏe các container
docker compose ps
# 5. Kiểm tra health check backend trực tiếp từ bên trong
curl.exe -i http://localhost:8088/actuator/health
```

7.1. Phương án Triển Khai Production Trên VPS Ubuntu

Nguyên tắc triển khai: Gắn thẻ bất biến campus-hub-backend:${RELEASE_ID} theo commit SHA, firewall chỉ mở port 80/443 (đóng 3306 và 8080), tự động sao lưu định kỳ volume mysql_data.

8. ĐÁNH GIÁ KẾT QUẢ THỰC HIỆN & HƯỚNG PHÁT TRIỂN

Tiêu chí	Hiện trạng trước	Kết quả đạt được (An Hải Dũng)
Thời gian	Cài đặt thủ công 45-60 phút	1 câu lệnh docker compose up -d trong 2 phút
Dung lượng	JDK nặng > 800MB; Node nặng > 1.2GB	Backend JRE ~250MB; Frontend Nginx chỉ ~35MB
Bảo mật	Chạy quyền root, port DB mở ngoài host	Non-root user UID 10001, DB nằm trong mạng cô lập (internal)
Độ tin cậy	Lỗi crash khi DB chưa sẵn sàng	Healthcheck probes tự động hóa, kiểm soát thứ tự phụ thuộc

8.1. Hướng phát triển trong tương lai

- **CI/CD Pipeline:** Tích hợp GitHub Actions tự động kiểm thử và build image lên GitHub Container Registry (GHCR).
- **Kubernetes (K8s) & Monitoring:** Chuyển đổi sang Kubernetes manifests và tích hợp Prometheus/Grafana giám sát tài nguyên.

NGƯỜI HƯỚNG DẪN / GIẢNG VIÊN
(Ký và ghi rõ họ tên)

.....

SINH VIÊN BÁO CÁO
(Ký và ghi rõ họ tên)

AN HẢI DŨNG